

Devdoots Assignment 05: Capstone Core Python Project

Team: Devdoots | **Team Leader:** Divyansh
Assigned Date: Friday, August 7, 2026
Submission Deadline: Sunday, August 9, 2026, before 9:00 PM IST

1. Project Overview & Objective

Welcome to the Capstone Assignment for Core Python! Over the past few weeks, team Devdoots has covered Python basics, Control Flow, Functions, Dynamic Arguments (*args & **kwargs), Scope, Lambda Functions, OOPs (Abstraction & Inheritance), Exception Handling, File I/O, and Modular Programming.

Assignment 05 is designed as a full-fledged, project-based assignment simulating an **Autonomous Swarm Telemetry & Control Engine (DAIVY Simulator)**. This project requires combining ALL concepts into a single, modular, and crash-proof Python application.

2. Key Concepts Covered in This Assignment

S.No.	Python Concept	Implementation Requirement in Project
1	OOPs (Abstraction & Inheritance)	Defining an abstract base class BaseCell using abc module and deriving DivyCell concrete class.
2	Dynamic Arguments (*args & **kwargs)	Handling variable telemetry parameters (e.g. voltage, temperature) and dynamic hyperparameter configs.
3	Custom Exception Handling	Creating user-defined exceptions (CellOverheatError, LowPowerError) and catching them gracefully.
4	Crash-Proof File I/O	Logging runtime telemetry into swarm_telemetry.txt in append mode and safely

S.No.	Python Concept	Implementation Requirement in Project
		reading logs using try-except.
5	Lambda Functions & Data Filtering	Using 1-liner Lambda functions to filter critical warnings and format telemetry percentages.
6	Modular Architecture & Imports	Splitting project into separate modules (exceptions.py, cells.py, logger.py, main.py).

3. Detailed Project Task Requirements

Task 1: Custom Exception Hierarchy (exceptions.py)

Create a dedicated file exceptions.py and define custom exceptions:

- SwarmError(Exception): Base custom exception class.
- CellOverheatError(SwarmError): Raised when cell temperature exceeds 85°C.
- LowPowerError(SwarmError): Raised when battery voltage/charge drops below 15%.

Task 2: Abstract Blueprint & Cell Implementation (cells.py)

Create cells.py and implement the core cell architecture:

- Define BaseCell(ABC) with abstract methods execute_pulse() and get_status().
- Inherit BaseCell into DivyCell:
 - Constructor __init__(self, cell_id, **config) to store configuration dictionaries dynamically.
 - Method log_telemetry(self, *metrics, **status) to format incoming telemetry tuples and status dictionaries.

Task 3: Telemetry Logger & Lambda Filter (logger.py)

Create logger.py to manage persistence:

- Implement SwarmLogger class to handle appending runtime logs to swarm_telemetry.txt.
- Include a method read_logs() protected by FileNotFoundError exception handling.
- Define a Lambda function format_percentage = lambda val: f"{val * 100:.1f}%" for display output.

Task 4: Main Simulation Controller (main.py)

In main.py, tie all modules together:

- Instantiate multiple DivyCell objects.
- Simulate cell operation loops where random voltage and temperature parameters are generated.
- Trigger custom exceptions when threshold conditions are crossed and handle them without crashing the program.

4. GitHub Submission Workflow & Rules

All team members must follow the strict TPM single-file/feature-branch workflow on GitHub:

Step	Git Command	Description
1. Sync Main Branch	git checkout main && git pull origin main	Ensure local repository is up to date with remote main branch.
2. Create Feature Branch	git checkout -b feature/username-assign5	Create a distinct feature branch (e.g., feature/divyansh-assign5).
3. Commit Changes	git add . && git commit -m "Complete Assignment 05"	Stage and commit code with meaningful message.
4. Push to Remote	git push origin feature/username-assign5	Push feature branch to GitHub repository.
5. Submit Pull Request	GitHub UI -> Create Pull Request	Submit PR for review before Sunday 9:00 PM IST deadline.

5. Evaluation & Review Criteria

- **Code Quality & Cleanliness:** Proper naming conventions, type safety, and no redundant code blocks.
- **Crash Immunity:** Program must handle missing files or abnormal data gracefully without breaking execution.
- **Modular Structure:** Clear separation into modules rather than writing everything in a single file.